

Forecasting Demand with a Neural Network in Google Colab

Problem → data → code, cell by cell — then judge the result like a forecaster.

Dr. Hyunwoong Ko | School of Manufacturing Systems and Networks | Fall 2026

Bring: a laptop, a free Google account, and the two files from Canvas — MFG510_NN_Forecasting_Colab.ipynb and demand_monthly.csv

Today's Big Picture

the “Forecasting as Learning” diagram, executed in six stages

1	Load & look	Cells 1–4	~10 min	read the CSV, plot it, name the pattern
2	Build the table	Cell 5	~10 min	the sliding window as three lines of code
3	Split by time	Cells 6–7	~5 min	train on the past, test on the future
4	Normalize & build	Cells 8–10	~10 min	the network from the lecture: $24 \rightarrow 64 \rightarrow 64 \rightarrow 1$
5	Train	Cells 10–12	~10 min	gradient descent, live; read the learning curve
6	Score & discuss	Cells 13–18	~20 min	MAD / MAPE vs. two baselines; then the challenge

Six cells have blanks (YOUR TURN) — you fill them; a CHECK cell after each tells you if you are right. Reveal follows on the next slide.

How Today Works

a guided lab: I frame it, you build it, we debrief together

0–12 min

Framing (me)

Slides 1–8: the problem, the data, setup, the notebook map, the three splits. Then I stop talking.

12–55 min

Build (you, in pairs)

Work through the notebook cell by cell. Fill each blank; the CHECK cell tells you if it is right. Use the Lab Guide on Canvas (every cell explained, with the discussion answers) whenever a cell is unclear. I circulate — raise a hand.

55–70 min

Debrief (together)

Answers to the blanks, the scorecard, the three discussion questions, and a live run of the challenge.

70–75 min

Wrap-up

Save your notebook, what is examinable, and what comes next. A follow-up assignment will be posted on Canvas.

Pace check. By minute 25 you should be at Cell 10; by minute 40 at Cell 13. Ahead of pace? Go on to the challenge (Cell 18).

Questions after class. Post on Piazza (link shared after class) — I answer as soon as possible, and answering a classmate earns the Piazza bonus. Office hours as usual.

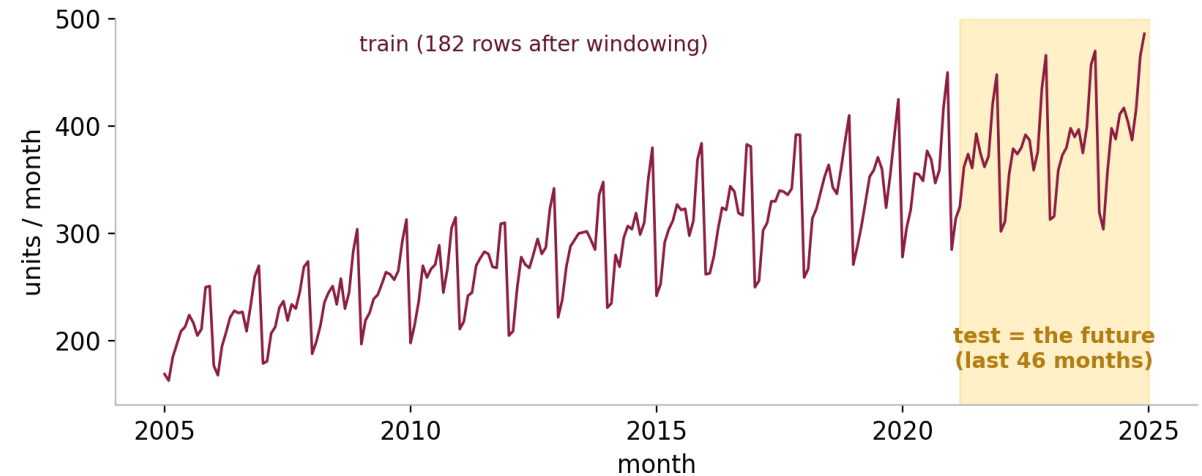
The Problem

one product family, twenty years of monthly demand

Setting. A plant has recorded monthly demand for one product family since January 2005 — 240 months. Planning needs a forecast for next month, every month, so it can set production and inventory (Lectures 2–6 consume this number).

The forecasting task. Given the last 12 months of demand and the calendar month, predict next month's demand: $\hat{A}_{t+1} = f(A_{t-11} \dots A_t, \text{month}; W)$.

How we judge it (Part 2 of the lecture): hold out the most recent months, forecast them, and compare MAD and MAPE against two free baselines — “last month” and “same month last year.”



What you should see: a rising level (trend), a repeating within-year rhythm (seasonality), and noise — the same three ingredients TES tracks. The gold band is the future we will forecast.

The series is synthetic — built with a known trend, seasonal profile, and noise — so that the answers are clean and the baselines are honest.

The Data — demand_monthly.csv

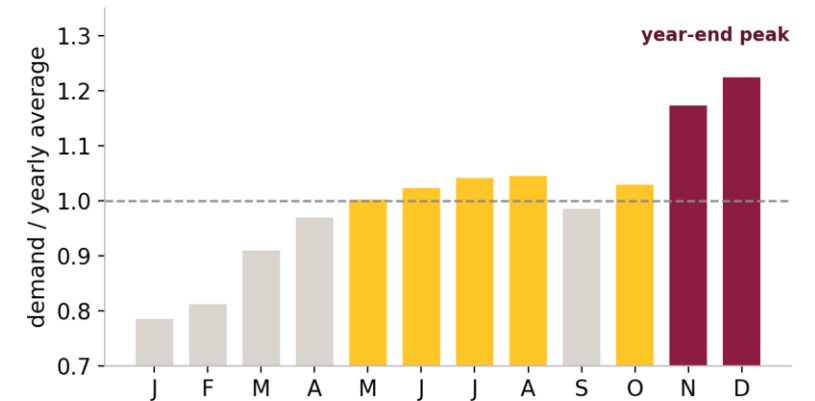
two columns, 240 rows: the whole input to today's lab

month	demand
2005-01	169
2005-02	163
2005-03	185
...	...
2024-11	465
2024-12	486

month = YYYY-MM (text) · demand = units shipped that month (integer)

Facts to keep in mind

- 240 months: 2005-01 to 2024-12
- range 163 – 486 units
- level rises about 0.8 units per month ($\approx 4\%/yr$)
- seasonal: Nov–Dec $\approx +20\%$, Jan–Feb $\approx -20\%$
- noise: roughly ± 8 units (1 s.d.)
- no missing values, no outliers



Seasonal profile: average demand in each calendar month relative to its year's mean.

What the network will see is NOT this table — it is a table we build from it in Stage 2: one row per month, 24 inputs and one label.

Compare with TES: TES would estimate a level, a trend, and 12 seasonal factors with three smoothing constants. The network will learn the same ingredients — but as 5,825 weights, from examples.

Getting Set Up — 5 Minutes

nothing to install: it all runs in the browser

1

Sign in colab.research.google.com — any free Google account (personal Gmail is fine if your ASU account blocks Colab).

2

Open the notebook File › Upload notebook → choose MFG510_NN_Forecasting_Colab.ipynb from Canvas.

3

Upload the data Run Cell 3 — an Upload button appears → choose demand_monthly.csv. (Or drag it into the Files panel on the left.)

4

Run cell by cell Click ▶ on each cell in order. Cells marked YOUR TURN have blanks (____) — fill them first; the CHECK cell after each prints ✓. (Run all stops at the first empty blank — expected.)

If something goes wrong

- “FileNotFoundError: demand_monthly.csv” → the file name must match exactly; re-run Cell 3.
- NameError or KeyError mentioning ____ → a blank is still empty.
- A CHECK cell fails → read its message; fix the blank above it and re-run both.
- Runtime disconnected → Runtime › Restart and run all (uploads are lost on restart — upload the CSV again).
- Different numbers from your neighbor → normal; training is stochastic.

Course policy reminder: this module’s case code is Python; exams assess models, algorithms, and pseudocode — not syntax.

The Notebook, Cell by Cell

a map to keep beside you

Cell	Stage	What the code does	What to look at
1–2	Install & imports	load pandas, numpy, matplotlib, TensorFlow/Keras	TensorFlow version prints — nothing to install
3–4	Load & clean	read the CSV; add a month index t ; plot	the demand plot: trend + season + noise
5 ★	Build the table	12 lag columns + 12 month one-hot columns (blanks: window length, label)	CHECK ✓ · shape (228, 28)
6 ★–7	Split by time	first 80% = train, last 20% = test (blanks: fraction, which slice is the past)	CHECK ✓ · 182 train 46 test
8–9	Normalize	a layer that rescales each input to mean 0, s.d. 1	one sample before / after
10 ★	Build the model	Normalization → Dense 64 → Dense 64 → Dense 1 (blanks: activation, outputs, loss)	CHECK ✓ · 5,825 trainable weights
11–12	Train	Adam optimizer, MAE loss, early stopping, 20% validation	the learning curve; “epochs run”
13 ★–15	Score	MAD & MAPE vs. two baselines (blanks: the MAD formula, the two baseline lags)	CHECK ✓ · the scorecard
16	(Optional) export	save the trained network to a file	skip if it errors
17–18 ★	Discuss & challenge	why the baseline is strong; retrain on the residual (blank: the residual target)	CHECK ✓ · baseline + network
19–20	Experiment (optional)	a provided helper: change ONE argument, rerun, compare	your experiment’s scorecard + curve

★ = YOUR TURN cell with blanks, followed by a CHECK cell

Stage 1 — Load & Look (Cells 1–4)

import the tools, read the file, plot before modeling

```
# Cell 2 — the toolkit
import numpy as np          # arrays & math
import pandas as pd         # tables (DataFrame)
import matplotlib.pyplot as plt # plots
import tensorflow as tf

from tensorflow import keras # the neural-network library
from tensorflow.keras import layers

# Cell 3 — read the CSV into a table
raw = pd.read_csv('demand_monthly.csv')

# Cell 4 — a month index, then look
dataset['t'] = np.arange(len(dataset))      # 0, 1, 2, ... 239
dataset['month_of_year'] = (dataset['t'] % 12) + 1
plt.plot(dataset['t'], dataset['demand'])
```

Reading the code

pandas DataFrame = a spreadsheet in memory. Columns are picked with `dataset['name']`; a new column is created by assigning to one.

`t` is the month index (0...239). `month_of_year` is `t mod 12`, plus 1 — the calendar month, 1–12.

keras is the part of TensorFlow that builds and trains networks; layers holds the building blocks.

Stop and look at the plot (Cell 4). Name the three ingredients out loud: trend, seasonality, noise. If you cannot see a pattern, no model will find one either.

Checkpoint: `raw.tail()` ends at 2024-12 with 486 units; after Cell 4 the table is 240 rows × 4 columns — month, demand, t, month_of_year; the plot shows trend + season + noise.

Stage 2 — Build the Training Table (Cell 5)

YOUR TURN

the sliding window is three lines of code — fill the two blanks

```
# twelve month switches: exactly one is 1, the rest 0
dataset = pd.get_dummies(dataset, columns=['month_of_year'],
                          prefix='m', dtype='float32')

# lag columns = the window, slid over the whole history
for k in range(1, ____):          # range(1, N) gives 1 ... N-1
    dataset[f'lag_{k}'] = dataset['demand'].shift(k)

dataset['label_next'] = dataset['____'] # the answer key
dataset = dataset.dropna() # first 12 rows have no full window
```

One row of the finished table (January 2006):

lag_1	lag_2	lag_3	...	lag_12	m_1	m_2	...	m_12	label
251	250	211	...	169	1	0	...	0	177

lag_1 = last month (Dec 2005 = 251) ... lag_12 = a year ago (Jan 2005 = 169) · m_1 = 1 says “this is January” · label = the demand we must predict (Jan 2006 = 177)

Checkpoint — the last line of Cell 5, `print(dataset.shape)`, prints (rows, columns) = (228, 28).

Why 228 rows: the first 12 months (all of 2005) have no full year behind them, so their lag cells are empty (NaN) and `dropna()` removes them → $240 - 12 = 228$. Why 28 columns: month + demand + t + 12 switches + 12 lags + label = 28 — only the 24 in the middle feed the network.

What just happened

`shift(k)` copies the demand column moved down by k rows — so in every row, `lag_k` is the demand k months earlier. Twelve shifts = the window, slid once over the entire history. Each row is one (question, answer) pair: the question is 24 numbers, the answer is that month’s demand.

`get_dummies` turns the month number into 12 one-hot switches — a category, not a quantity (December is not $12 \times$ January).

Blank 1: the window holds how many months? Mind that `range(1, N)` stops at $N-1$. Blank 2: what are we predicting?

Stage 2 — Build the Training Table (Cell 5)

ANSWERS

the sliding window is three lines of code — answers

```
# twelve month switches: exactly one is 1, the rest 0
dataset = pd.get_dummies(dataset, columns=['month_of_year'],
                          prefix='m', dtype='float32')

# lag columns = the window, slid over the whole history
for k in range(1, 13):      # range(1, N) gives 1 ... N-1
    dataset[f'lag_{k}'] = dataset['demand'].shift(k)

dataset['label_next'] = dataset['demand'] # the answer key
dataset = dataset.dropna() # first 12 rows have no full window
```

One row of the finished table (January 2006):

lag_1	lag_2	lag_3	...	lag_12	m_1	m_2	...	m_12	label
251	250	211	...	169	1	0	...	0	177

lag_1 = last month (Dec 2005 = 251) ... lag_12 = a year ago (Jan 2005 = 169) · m_1 = 1 says “this is January” · label = the demand we must predict (Jan 2006 = 177)

Checkpoint — the last line of Cell 5, `print(dataset.shape)`, prints (rows, columns) = (228, 28).

Why 228 rows: the first 12 months (all of 2005) have no full year behind them, so their lag cells are empty (NaN) and `dropna()` removes them → $240 - 12 = 228$. Why 28 columns: month + demand + $t + 12$ switches + 12 lags + label = 28 — only the 24 in the middle feed the network.

What just happened

`shift(k)` copies the demand column moved down by k rows — so in every row, `lag_k` is the demand k months earlier. Twelve shifts = the window, slid once over the entire history. Each row is one (question, answer) pair: the question is 24 numbers, the answer is that month’s demand.

`get_dummies` turns the month number into 12 one-hot switches — a category, not a quantity (December is not $12 \times$ January).

Blank 1 = 13 (range stops one short, so $1 \dots 12$ = twelve lags). Blank 2 = demand — the label is what we predict. CHECK cell: (228, 28).

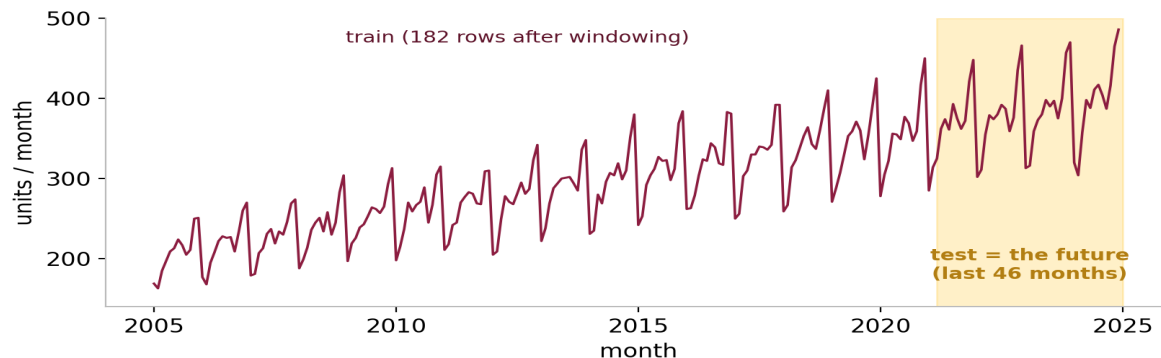
Stage 3 — Split by Time (Cells 6–7)

YOUR TURN

train on the past, test on the future — fill the three blanks

```
# Cell 6 – the most recent 20% of months is the test set
split = int(len(dataset) * _____)
train_dataset = dataset.iloc[_____] # the past
test_dataset = dataset.iloc[_____] # the future

# Cell 7 – inputs (x) vs. label (y) [provided]
feature_cols = [f'lag_{k}' for k in range(1, 13)] + \
               [c for c in dataset.columns if c.startswith('m_')]
train_features = train_dataset[feature_cols] # 182 x 24
train_labels = train_dataset['label_next'] # 182
```



Why not a random split?

In a random split the network would be tested on 2015 while having trained on 2014 and 2016 — it has seen the neighbors. That is not forecasting.

By holding out the LAST 46 months we test exactly what planning needs: predicting a future the model never saw — including a demand level higher than anything in training.

`iloc[:split]` = rows before the cut; `iloc[split:]` = rows from the cut on.
Which one is the past?

Checkpoint: “182 train months | 46 test months” and the CHECK cell confirms test comes AFTER train.

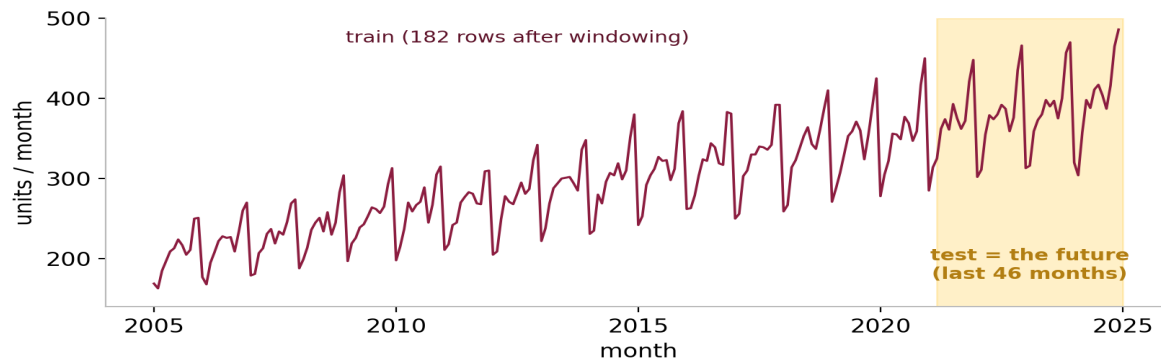
Stage 3 — Split by Time (Cells 6–7)

ANSWERS

train on the past, test on the future — answers

```
# Cell 6 – the most recent 20% of months is the test set
split = int(len(dataset) * 0.8)
train_dataset = dataset.iloc[:split]      # the past
test_dataset  = dataset.iloc[split:]      # the future

# Cell 7 – inputs (x) vs. label (y) [provided]
feature_cols = [f'lag_{k}' for k in range(1, 13)] + \
               [c for c in dataset.columns if c.startswith('m_')]
train_features = train_dataset[feature_cols]  # 182 x 24
train_labels   = train_dataset['label_next']  # 182
```



Why not a random split?

In a random split the network would be tested on 2015 while having trained on 2014 and 2016 — it has seen the neighbors. That is not forecasting.

By holding out the LAST 46 months we test exactly what planning needs: predicting a future the model never saw — including a demand level higher than anything in training.

$0.8 \cdot \text{iloc}[:\text{split}]$ = rows before the cut = the past = train
 $\cdot \text{iloc}[\text{split}:]$ = the future = test.

Checkpoint: “182 train months | 46 test months” and the CHECK cell confirms test comes AFTER train.

Three Splits, Three Jobs

the 240 months end up in three blocks — each with exactly one role



time → chronological blocks: each block is the "future" of the one before it

TRAIN · 145 rows

Cells 6-7 · 11

Gradient descent sees only these rows. Every weight update comes from here — this is the "answer key" the network studies.

VALIDATION · 37 rows

Cell 11: `validation_split = 0.2`

The last 20% of the training block, never used for updates. Scored after every epoch → the gold curve in Cell 12. Early stopping watches it to decide WHEN to stop and which weights to keep.

TEST · 46 rows

Cells 6 · 13-15

The future. Touched exactly once, at the end, for the scorecard. It never influences the weights or the stopping rule — otherwise the score would be flattery, not forecasting.

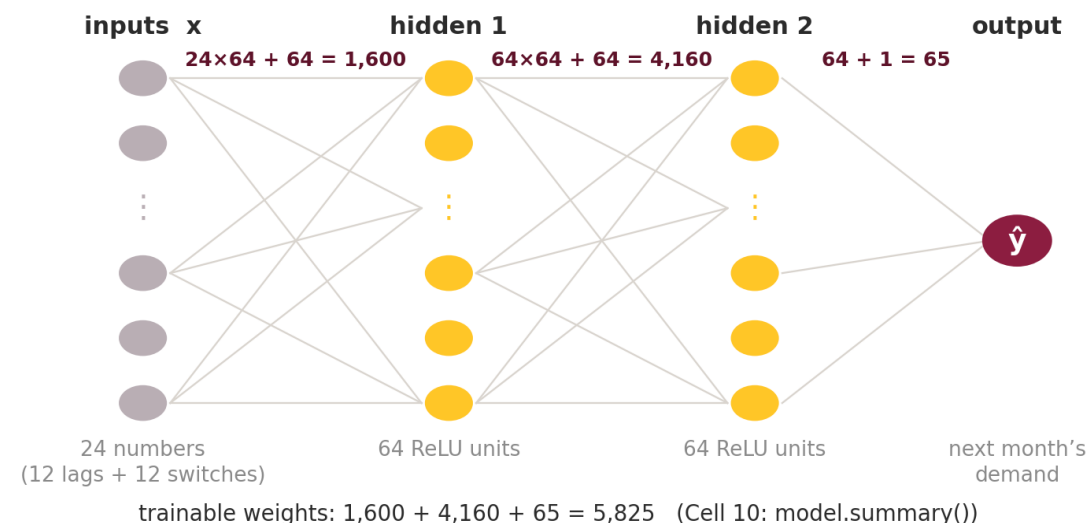
Stage 4 — Normalize & Build the Network (Cells 8–10)

YOUR TURN

the lecture's architecture, in four lines — fill the three blanks

```
# Cell 8 — rescale every input to mean 0, s.d. 1 [provided]
normalizer = layers.Normalization(axis=-1)
normalizer.adapt(np.array(train_features)) # stats from TRAIN only

# Cell 10 — the network, layer by layer
model = keras.Sequential([
    normalizer, # 24 inputs, rescaled
    layers.Dense(64, activation='____'), # hidden layer 1
    layers.Dense(64, activation='____'), # hidden layer 2
    layers.Dense(____), # output
])
```



Normalization = the “scaled by 100” from the hand example. Lags are in the hundreds, switches are 0/1 — without rescaling the lags would dominate every weighted sum and gradient descent would crawl. `adapt()` learns the means and s.d. from the training rows only, so no information leaks from the test months.

Dense = the perceptron slides, stacked. `Dense(64, ...)`: 64 units, each computing $g(\mathbf{w}^T \mathbf{x} + b)$. Blank 1: which g did the hand-worked example use? Blank 2: a network that predicts one quantity needs how many output units? (The output layer has no activation — why?)

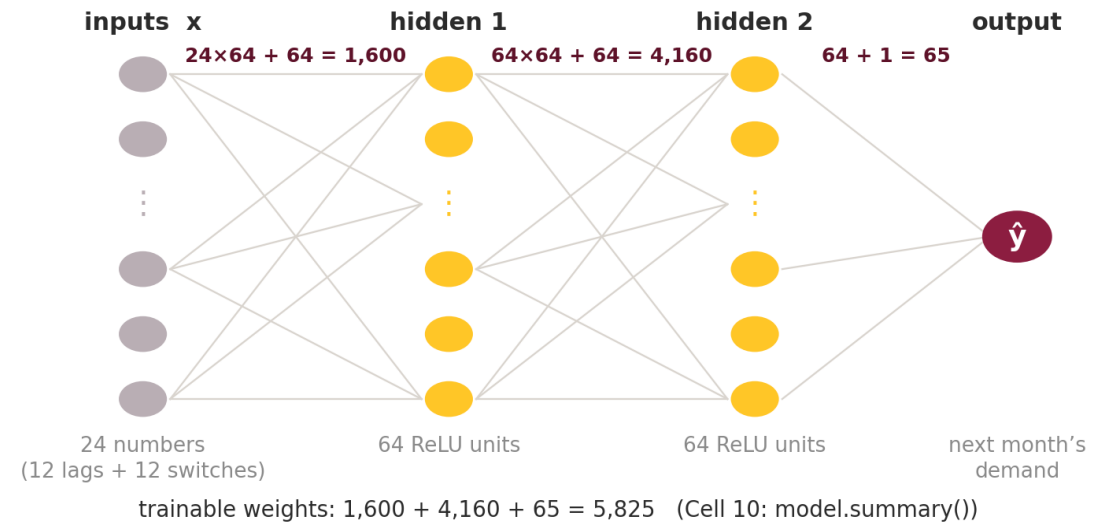
Stage 4 — Normalize & Build the Network (Cells 8–10)

ANSWERS

the lecture's architecture, in four lines — answers

```
# Cell 8 — rescale every input to mean 0, s.d. 1 [provided]
normalizer = layers.Normalization(axis=-1)
normalizer.adapt(np.array(train_features)) # stats from TRAIN only

# Cell 10 — the network, layer by layer
model = keras.Sequential([
    normalizer, # 24 inputs, rescaled
    layers.Dense(64, activation='relu'), # hidden layer 1
    layers.Dense(64, activation='relu'), # hidden layer 2
    layers.Dense(1), # output
])
```



Normalization = the “scaled by 100” from the hand example. Lags are in the hundreds, switches are 0/1 — without rescaling the lags would dominate every weighted sum and gradient descent would crawl. `adapt()` learns the means and s.d. from the training rows only, so no information leaks from the test months.

Answers. `relu` — the activation from the hand-worked example, $g(z) = \max(0, z)$, the source of non-linearity. 1 — one output unit, because we predict one quantity; it has no activation: a plain weighted sum. The CHECK cell counts the weights: $24 \times 64 + 64 + 64 \times 64 + 64 + 64 + 1 = 5,825$.

Stage 5 — Train (Cells 10–12)

YOUR TURN

gradient descent, live — fill the last blank of Cell 10

```
# Cell 10 (end) — choose the loss and the optimizer
model.compile(optimizer=keras.optimizers.Adam(0.01), # step size
              loss='___') # the objective J(W)

# Cell 11 — run gradient descent, with early stopping [provided]
early_stop = keras.callbacks.EarlyStopping(monitor='val_loss',
                                           patience=40, restore_best_weights=True)
history = model.fit(train_features, train_labels,
                   epochs=300, validation_split=0.2,
                   callbacks=[early_stop])
```

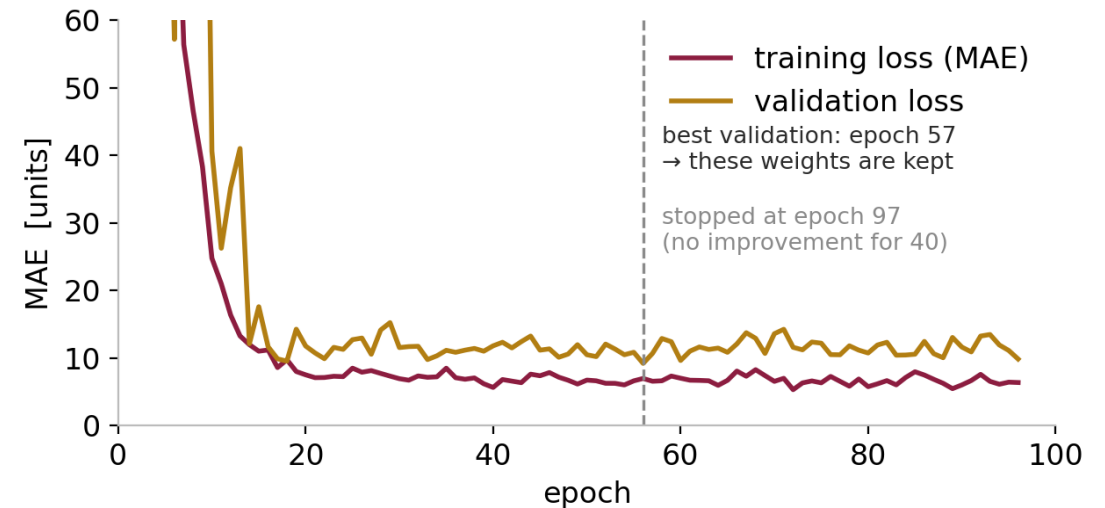
Mapping to the lecture

loss = the objective $J(W)$: we want gradient descent to minimize the forecaster's MAD — which Keras loss is that?

optimizer = Adam, an adaptive-learning-rate gradient descent; 0.01 is the learning rate η
epochs = up to 300 — one epoch = one pass over the 145 training rows in mini-batches of 32 (5 weight updates)

validation_split = 0.2 — the last 37 training rows are set aside (never trained on) and scored after every epoch — see “Three Splits, Three Jobs”

EarlyStopping — stop once validation has not improved for 40 epochs and keep the best weights: Regularization 2 from the lecture, live



Reading the curve (Cell 12)

Both lines fall fast, then flatten — the loss-surface walk from the slides. The validation line sits above the training line: that gap is memorization. The dashed line marks the best validation epoch — those weights are the ones kept; training stops 40 epochs later because nothing improved. That is early stopping doing its job.

Stage 5 — Train (Cells 10–12)

ANSWERS

gradient descent, live — answer

```
# Cell 10 (end) — choose the loss and the optimizer
model.compile(optimizer=keras.optimizers.Adam(0.01), # step size
              loss='mean_absolute_error') # the objective J(W)

# Cell 11 — run gradient descent, with early stopping [provided]
early_stop = keras.callbacks.EarlyStopping(monitor='val_loss',
                                           patience=40, restore_best_weights=True)
history = model.fit(train_features, train_labels,
                   epochs=300, validation_split=0.2,
                   callbacks=[early_stop])
```

Mapping to the lecture

loss = mean_absolute_error — our MAD as the objective $J(W)$ (mean_squared_error would be MSD)

optimizer = Adam, an adaptive-learning-rate gradient descent; 0.01 is the learning rate η
epochs = up to 300 — one epoch = one pass over the 145 training rows in mini-batches of 32 (5 weight updates)

validation_split = 0.2 — the last 37 training rows are set aside (never trained on) and scored after every epoch — see “Three Splits, Three Jobs”

EarlyStopping — stop once validation has not improved for 40 epochs and keep the best weights: Regularization 2 from the lecture, live



Reading the curve (Cell 12)

Both lines fall fast, then flatten — the loss-surface walk from the slides. The validation line sits above the training line: that gap is memorization. The dashed line marks the best validation epoch — those weights are the ones kept; training stops 40 epochs later because nothing improved. That is early stopping doing its job.

Stage 6 — Score Like a Forecaster (Cells 13–15)

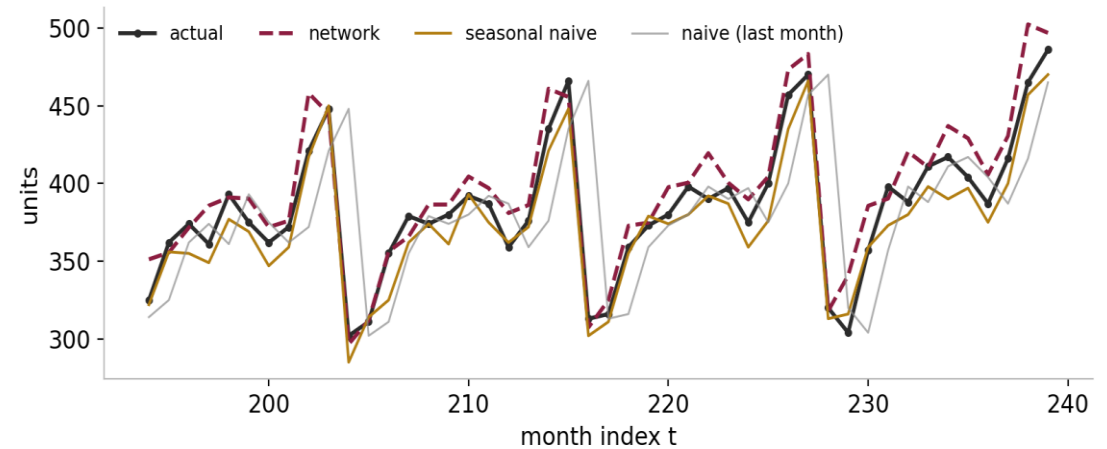
YOUR TURN

MAD and MAPE, against two baselines that cost nothing — fill the three blanks

```
# Cell 13 — forecast the 46 held-out months
pred = model.predict(test_features).flatten()
actual = test_labels.values

def mad(a, f): return _____ # Part 2
def mape(a, f): return np.mean(np.abs((f - a) / a)) * 100

# two free baselines, straight from the lag columns
naive = test_dataset['_____'].values # copy last month
snaive = test_dataset['_____'].values # copy same month last year
```



Why baselines? “MAD = 14” means nothing on its own. A forecast earns its keep only by beating what you could have done for free. Blank 1: write MAD as code — the mean of $|f - a|$ (`np.mean`, `np.abs`). Blanks 2–3: which lag column is “last month”, and which is “the same month last year”?

Reading the plot (Cell 14)

Black = actual. The network (dashed maroon) follows the seasonal swings and the rising level. Seasonal naive (gold) is simply last year’s curve — it tracks the shape but runs a little low, because it cannot see the trend. Naive (gray) lags every turn by a month.

Stage 6 — Score Like a Forecaster (Cells 13–15)

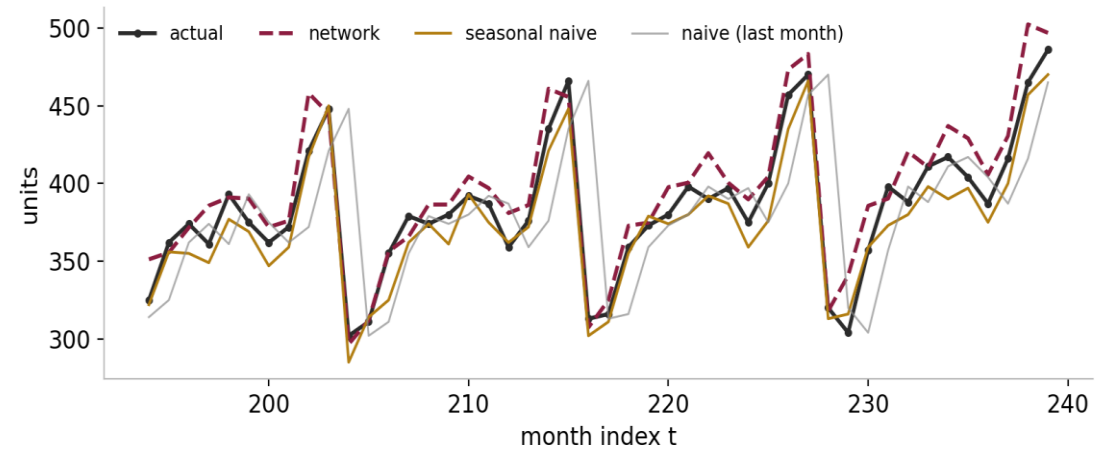
ANSWERS

MAD and MAPE, against two baselines that cost nothing — answers

```
# Cell 13 — forecast the 46 held-out months
pred = model.predict(test_features).flatten()
actual = test_labels.values

def mad(a, f): return np.mean(np.abs(f - a)) # Part 2
def mape(a, f): return np.mean(np.abs((f - a) / a)) * 100

# two free baselines, straight from the lag columns
naive = test_dataset['lag_1'].values # copy last month
snaive = test_dataset['lag_12'].values # copy same month last year
```



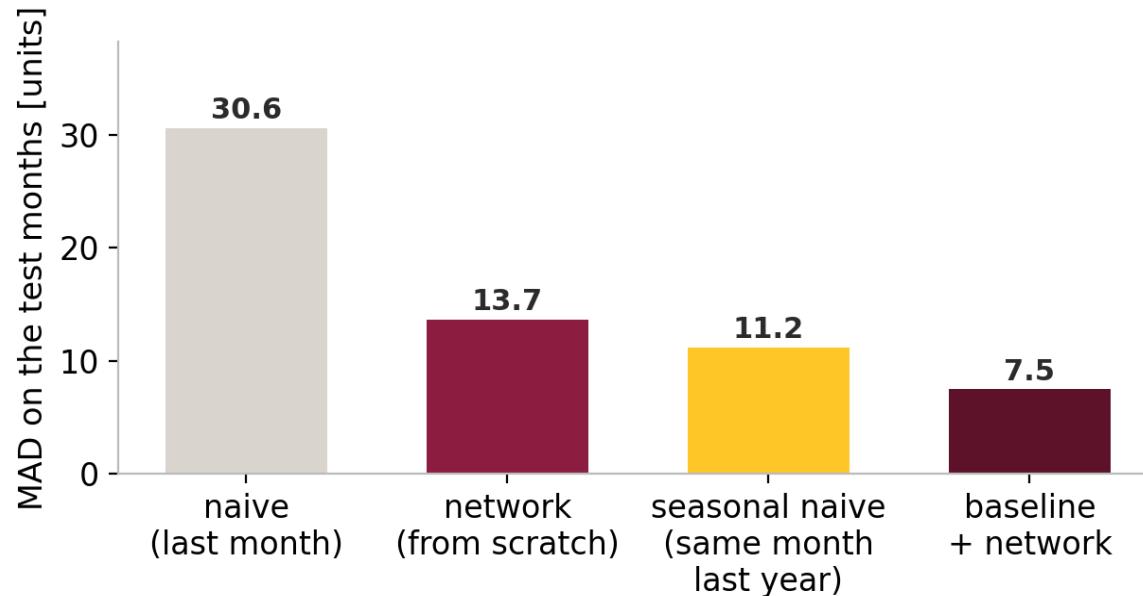
Answers. $MAD = \text{np.mean}(\text{np.abs}(f - a))$ — the Part 2 formula, one line. Naive = lag_1: last month's demand is this month's forecast. Seasonal naive = lag_12: the same month last year. The CHECK cell knows their MADs: ≈ 30.6 and ≈ 11.2 .

Reading the plot (Cell 14)

Black = actual. The network (dashed maroon) follows the seasonal swings and the rising level. Seasonal naive (gold) is simply last year's curve — it tracks the shape but runs a little low, because it cannot see the trend. Naive (gray) lags every turn by a month.

The Scorecard

what you should see — approximately



Forecast	MAD (units)	MAPE
Naive — copy last month	≈ 30	≈ 8%
Network — from scratch	≈ 12–16	≈ 3–4%
Seasonal naive — same month last year	≈ 11	≈ 3%
Baseline + network (Cell 18)	≈ 8–9	≈ 2%

Your numbers will differ a little: training starts from random weights and runs in a random order. Rank, not decimals, is what matters.

The network beats “copy last month” by a factor of two — and a free one-line baseline still edges it out. Both facts are the lesson.

Say it the way Part 2 taught you: “On the 46 held-out months, the network’s MAPE is about 3.5% — within a few units of the seasonal naive and less than half the naive error.”

Discussion — Why Is the Free Baseline So Strong? (Cell 17)

five minutes in your group, then we compare answers

1

Why does seasonal naive do so well on THIS series?

Hint: how was the series built? What does “same month last year” assume — and is that assumption almost exactly true here?

2

The test months are the future: their level is higher than anything in training. Who suffers?

Hint: seasonal naive carries last year’s level forward automatically. What does a network do with inputs outside the range it was trained on?

3

We trained on 182 rows. Is that a lot for 5,825 weights?

Hint: compare with the regression examples — 5 or 6 points for 2 parameters felt thin. What ratio do we have here?

The lesson of Part 2, in practice: a model is only as good as the baseline it beats — and the baseline must be the strongest free one you can think of.

Challenge — Stand on the Baseline’s Shoulders (Cell 18)

YOUR TURN

same network, same inputs — a smarter question — fill the blank

$$\hat{A}_t = A_{t-12} + f(x_t; W) \quad \text{seasonal naive} + \text{the correction the network learns}$$

```
# the target is what seasonal naive gets WRONG
res_train = train_labels - train_dataset['_____']

model_res = keras.Sequential([normalizer,
    layers.Dense(64, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)])
model_res.compile(optimizer=keras.optimizers.Adam(0.01),
    loss='mean_absolute_error')
model_res.fit(train_features, res_train, epochs=300,
    callbacks=[early_stop], validation_split=0.2)

pred_res = snaive + model_res.predict(test_features).flatten()
```

Blank: the residual is “demand minus the seasonal-naive forecast.” Which lag column is the seasonal naive? (The CHECK cell expects a small residual.)

What changed — and what didn’t

Nothing about the network: same 24 inputs, same layers, same optimizer. Only the label changed: instead of “predict demand,” it is “predict how far last year’s value will be off.”

The baseline handles level and season; the network only has to learn the drift and whatever pattern is left — a much smaller, easier target.

This is the lecture’s line made real: “associative models are also used to correct the time-series models.” Practitioners almost always ask a network to fix a baseline, not to start from zero.

Challenge — Stand on the Baseline’s Shoulders (Cell 18)

ANSWERS

same network, same inputs — a smarter question — answer

$$\hat{A}_t = A_{t-12} + f(x_t; W) \quad \text{seasonal naive} + \text{the correction the network learns}$$

```
# the target is what seasonal naive gets WRONG
res_train = train_labels - train_dataset['lag_12']

model_res = keras.Sequential([normalizer,
                              layers.Dense(64, activation='relu'),
                              layers.Dense(64, activation='relu'),
                              layers.Dense(1)])
model_res.compile(optimizer=keras.optimizers.Adam(0.01),
                  loss='mean_absolute_error')
model_res.fit(train_features, res_train, epochs=300,
              callbacks=[early_stop], validation_split=0.2)

pred_res = snaive + model_res.predict(test_features).flatten()
```

lag_12 — the residual is demand minus the seasonal-naive forecast. Typical result: MAD ≈ 8–9, the first model in the notebook to beat the seasonal naive, reliably.

What changed — and what didn’t

Nothing about the network: same 24 inputs, same layers, same optimizer. Only the label changed: instead of “predict demand,” it is “predict how far last year’s value will be off.”

The baseline handles level and season; the network only has to learn the drift and whatever pattern is left — a much smaller, easier target.

This is the lecture’s line made real: “associative models are also used to correct the time-series models.” Practitioners almost always ask a network to fix a baseline, not to start from zero.

Code Vocabulary — the 10 Words You Met Today

enough Python to read the notebook; none of it is on the exam

Word	Library	What it does	Lecture concept
DataFrame	pandas	a table in memory; columns by name: df['demand']	the spreadsheet
shift(k)	pandas	the same column moved down k rows	lag $k = A_{t-k}$
get_dummies	pandas	category $\rightarrow$ one-hot 0/1 columns	seasonal switches; dummy variables in regression
iloc[a:b]	pandas	rows a to b-1 by position	the time split
Normalization	keras	rescale inputs to mean 0, s.d. 1	“scale by 100”
Sequential	keras	a network as an ordered list of layers	the architecture figure
Dense(n, relu)	keras	n units of $g(w^T x + b)$, $g = \text{ReLU}$	the perceptron, n times
compile	keras	pick the loss J and the optimizer	MAD as objective; gradient descent
fit	keras	run gradient descent for some epochs	training loop; backprop inside
predict	keras	forward pass on new rows	the hand-worked example

Wrap-Up

what you did, what to keep, what comes next

What you did today

- turned one time series into a learning table
- built and trained the lecture's network
- read a learning curve
- scored a forecast against honest baselines
- used a network to correct a baseline

Keep your notebook

File › Save a copy in Drive.

A follow-up assignment will build on this notebook; details and the due date will be posted on Canvas.

The final group project then extends the same notebook: classical methods vs. the network.

What is examinable

- the forward pass (by hand)
- MSE / MAE loss, gradient descent, the learning-rate trade-off
- what backprop computes
- overfitting, early stopping
- windowing & one-hot inputs, and why the split is by time

Not: Python syntax.

Next class: Linear Programming — the forecast becomes the demand row of an optimization model.